Geodesic Distances: the Mathematics of `geodesic.py`

November 2026

This note states every formula that `geodesic.py` (in `Dubins_TSP_New_Implementation_claude/`) computes, in the notation of `Dubins_TSP.pdf`. The file gives the geodesic distance of eq. (9) of the paper, the matrix D_{geo} of Step 1 of Section 0.9, and the obstacle-free paths that Step 2 of Section 0.11 builds on. Beside each symbol is the name it has in the code, and each section names the functions it describes. The statements about the inflated polygons were checked on 3 000 random obstacles made by `scenario.py`, and those about the distances on the maps of seeds 1–10; the numbers are quoted where they are used.

1 Obstacles, Terminals and Margins

The field contains m disjoint polygonal obstacles $O_1, \dots, O_m$, the base station $S_0 = (x_0, y_0)$ and the sensors $S_i = (x_i, y_i)$, $i = 1, \dots, n$. The base and the sensors are the $n_t = n + 1$ *terminals*, numbered $0, 1, \dots, n$ as in the paper. (In the code, terminal 0 is the base and terminal i is `scen["xy"][i - 1]`, the sensor with the 0-based id $i - 1$.)

The paper defines the free space by inflating the obstacles by a safety clearance, eqs. (4) and (5):

$$F = \mathbb{R}^2 \setminus \bigcup_{j=1}^{m} \operatorname{int}\big(O_j^{\text{inflated}}\big), \qquad r_{\text{safe}} = \frac{w}{2} + \delta_{\text{margin}}. \tag{1}$$

`geodesic.py` works with three margins:

$$r_{\text{safe}} = 2\,\text{m}, \qquad r_{\text{term}} = r_{\text{safe}} + 0.5\,\text{m} = 2.5\,\text{m}, \qquad r_{\text{plan}} = r_{\text{safe}} + R = 7\,\text{m}, \tag{2}$$

with $R = L/\tan\delta_{\max} = 5\,\text{m}$ from eq. (7) and the defaults of `config.py`. Lines between two obstacle corners keep the large margin r_{plan}, which leaves the sink room to turn. Lines that start or end at a terminal keep only r_{term}, because sensors may lie closer to an obstacle than r_{plan}: on seeds 1–10, 577 of the 1 015 sensors do. The margin r_{safe} itself is not used here; the clearance of the final trajectory is measured against it later (Section 6). r_{term} has no name in the code, which writes `p["r_safe"] + 0.5`. Table 1 lists the symbols.

2 Inflated Obstacles

The function `inflate` grows each obstacle by a distance r. The exact grown obstacle is the set of points within r of it, the Minkowski sum with the disc $\mathcal{D}_r$ of radius r:

$$O_k \oplus \mathcal{D}_r = \big\{x \in \mathbb{R}^2 : \operatorname{dist}(x, O_k) \le r\big\}. \tag{3}$$

Its boundary has round arcs at the convex corners. The code uses a polygon $P_r(O_k)$ in its place: the shapely `buffer` with `quad_segs` $= q = 2$, whose boundary is built as follows (Figure 1).

Edges. Each edge of O_k is moved outward, parallel to itself, by the distance r.

Convex corners. Let the boundary of O_k turn through the angle $\theta \in (0, \pi)$ at a convex corner c: θ is the angle between the outward normals of the edge that ends at c and the edge that starts there, and the moved edges end on the circle of radius r about c. The gap between them is closed by

$$n_\theta = \max\Big(1,\ \Big\lfloor \frac{\theta}{\pi/(2q)} + \frac{1}{2} \Big\rfloor\Big) = \max\Big(1,\ \Big\lfloor \frac{4\theta}{\pi} + \frac{1}{2} \Big\rfloor\Big) \quad \text{chords, each turning through} \quad \Delta = \frac{\theta}{n_\theta}, \tag{4}$$

that is, the arc is cut into steps of about 45°. The chord ends are the points

$$c + r\,\big(\cos(\varphi_0 + j\Delta),\ \sin(\varphi_0 + j\Delta)\big), \qquad j = 0, 1, \dots, n_\theta, \tag{5}$$

where φ_0 is the direction of the outward normal of the edge that ends at c, and the angle grows in the direction in which the boundary turns. A right-angled corner, for example, gets two chords of 45°.

Table 1: Symbols and their names in `geodesic.py`.

Symbol	Code	Meaning
O_k, $k = 1, \dots, m$	`scen["obstacles"][k - 1]`	obstacle polygon (its vertices)
S_0	`scen["base"]`, terminal 0	base station
S_i, $i = 1, \dots, n$	`scen["xy"][i - 1]`, terminal i	sensor
n_t	`n_terminals`, `n_t`	number of terminals, $n + 1$
$r_{\text{safe}}, R, r_{\text{plan}}$	`p["r_safe"], p["R"], p["r_plan"]`	margins and turning radius, eq. (2)
r_{term}	`p["r_safe"] + 0.5`	margin of lines that touch a terminal
$P_r(O_k)$	`inflate(obstacles, r)`	O_k inflated by r, as a polygon (Section 2)
q	`quad_segs=2`	chords per quarter circle at the corners
ε	`1e-6`	depth by which a line may enter an inflated polygon
I_r	`_inside(inflate(obstacles, r))`	union of the inflated polygons, shrunk by ε
$v_0, \dots, v_{N-1}$	`nodes`	graph nodes: the terminals, then the corners of the $P_{r_{\text{plan}}}(O_k)$
E, w_{ab}	`graph`	edges of the visibility graph and their lengths
$d_a(v)$, $\text{pred}_a(v)$	`dist[a, v]`, `pred[a, v]`	shortest distance from terminal a to node v, and the node before v
D	`D`	geodesic distances between terminals, an $n_t \times n_t$ matrix
Γ_{ab}	`path(geo, a, b)`	shortest obstacle-free polyline from terminal a to terminal b
O	`obstacles`	union of the original obstacles, for clearance checks

Reflex corners. At a corner where the boundary turns the other way (the inside corner of an L), the two moved edges cross, and their crossing point is the vertex.

Overlaps. Where the moved boundary overlaps itself, as in the pocket of a C or the notches of a cross, the polygon is the union of the pieces, so the covered parts of the boundary disappear.

How close the polygon comes to the obstacle. A chord that turns through Δ comes within $r\cos(\Delta/2)$ of its corner, at its midpoint. By eq. (4), $n_\theta = 1$ needs $\theta < 3\pi/8$, and $n_\theta = n \geq 2$ needs $\theta < (n + \frac{1}{2})\pi/4$, so

$$\Delta < \frac{3\pi}{8} = 67.5^\circ, \qquad r\cos\frac{\Delta}{2} > r\cos\frac{3\pi}{16} = 0.8315\,r. \tag{6}$$

No vertex lies farther than r from O_k, and every chord lies inside the disc of its corner, so

$$\boxed{O_k \oplus \mathcal{D}_{0.8315\,r} \;\subseteq\; P_r(O_k) \;\subseteq\; O_k \oplus \mathcal{D}_r.} \tag{7}$$

For $r_{\text{term}} = 2.5\,\text{m}$ the polygon keeps at least 2.08 m from the obstacle, more than $r_{\text{safe}} = 2\,\text{m}$; for $r_{\text{plan}} = 7\,\text{m}$ it keeps at least 5.82 m. Since $0.8315\,r_{\text{plan}} > r_{\text{term}}$, every $P_{r_{\text{term}}}(O_k)$ lies inside $P_{r_{\text{plan}}}(O_k)$.

Check. On 3 000 random obstacles of `scenario.py` (random polygons, L, C and cross shapes), inflated by 2.5 m and by 7 m, every one of the 29 047 convex corners whose arc lies on the boundary has exactly the vertices of eq. (5); at the other 6 119 corners another arm of the same obstacle covers part of the arc. The largest chord angle was 67.44°, the closest approach of a boundary to its obstacle $0.8318\,r$, and the farthest vertex lay at r to within $10^{-11}r$.

3 The Visibility Test

The function `_inside` shrinks each inflated polygon by $\varepsilon = 10^{-6}\,\text{m}$ and takes the union. For a margin r,

$$I_r = \bigcup_{k=1}^{m} \big(P_r(O_k) \ominus \mathcal{D}_\varepsilon\big), \qquad A \ominus \mathcal{D}_\varepsilon = \big\{x : \text{the disc of radius } \varepsilon \text{ about } x \text{ lies in } A\big\}. \tag{8}$$

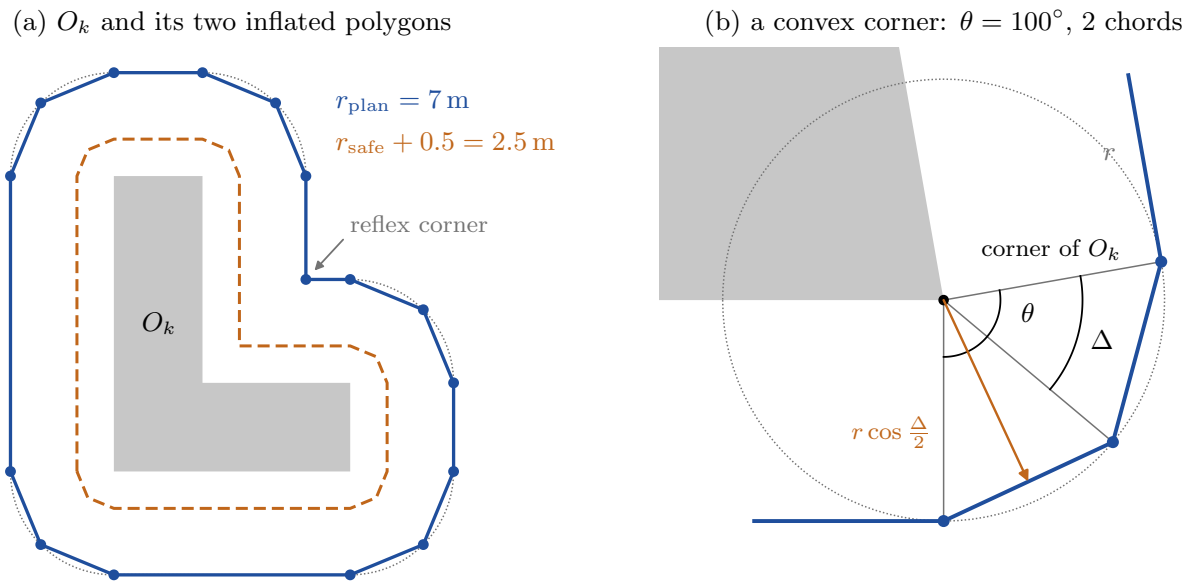


Figure 1: (a) An L-shaped obstacle, 16 m by 20 m with 6 m wide arms, and its polygons $P_{r_{\text{plan}}}$ (blue, 12 vertices, which become graph nodes) and $P_{r_{\text{term}}}$ (orange, dashed), drawn by `geodesic.inflate`. The dotted line is the exact grown obstacle $O_k \oplus \mathcal{D}_{r_{\text{plan}}}$. At the reflex corner the two moved edges meet. (b) A convex corner where the boundary turns through $\theta = 100°$: eq. (4) gives 2 chords of $\Delta = 50°$, whose ends lie on the circle of radius r, and the chords come within $r \cos 25° = 0.906\,r$ of the corner.

A straight line segment $[p, q]$ is *clear at margin* r if

$$[p, q] \cap I_r = \emptyset. \tag{9}$$

Shrinking by ε removes a strip of width ε along each boundary, so a clear segment may touch an inflated polygon or run along its boundary, but not enter it by more than ε. By eq. (7), a segment that is clear at margin r keeps at least $0.8315\,r - \varepsilon$ from every obstacle.

4 The Visibility Graph

Nodes. The function `build` takes the terminals, followed by the corners of the inflated obstacles:

$$\begin{gathered} V = \{v_0, \ldots, v_{N-1}\}, \qquad v_a = S_a \quad (a = 0, \ldots, n), \\ v_{n_t}, \ldots, v_{N-1} = \text{the vertices of the outer boundaries of } P_{r_{\text{plan}}}(O_1), \ldots, P_{r_{\text{plan}}}(O_m). \end{gathered} \tag{10}$$

If a polygon had a hole, the corners of the hole would not be nodes; none of the 108 polygons on seeds 1–10 has one.

Edges. Every pair of nodes $a < b$ is tested once. Because $a < b$, the pair touches a terminal exactly when $a < n_t$:

$$\boxed{(a, b) \in E \iff \begin{cases} [v_a, v_b] \cap I_{r_{\text{term}}} = \emptyset & \text{if } a < n_t \text{ (the line starts or ends at a terminal),} \\ [v_a, v_b] \cap I_{r_{\text{plan}}} = \emptyset & \text{if } a \geq n_t \text{ (the line joins two corners).} \end{cases}} \tag{11}$$

If there are no obstacles, every pair is an edge. Each edge has its straight-line length as its weight:

$$w_{ab} = \|v_a - v_b\| = \sqrt{(x_a - x_b)^2 + (y_a - y_b)^2}. \tag{12}$$

The graph is undirected, so (a, b) and (b, a) are the same edge. $N(N-1)/2$ pairs are tested; on seed 1 there are 99 terminals and 148 corner nodes, so 30 381 pairs, of which 7 898 are edges (6 800 touching a terminal and 1 098 between corners).

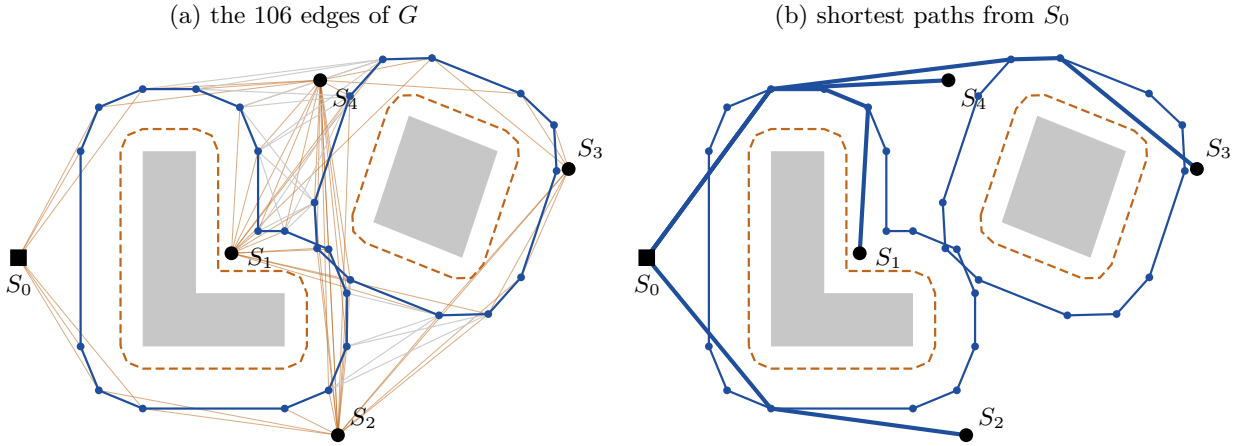


Figure 2: The graph of `geodesic.build` on a small map: two obstacles, the base S_0 and four sensors. Blue solid: $P_{r_{\text{plan}}}$ and its 28 corner nodes; orange dashed: $P_{r_{\text{term}}}$. (a) Of the 528 pairs of nodes, 106 are joined: 61 edges touch a terminal and are tested at margin r_{term} (orange), and 45 join two corners and are tested at margin r_{plan} (grey). (b) The shortest path from S_0 to each sensor. S_1 lies in the inside corner of the L, inside $P_{r_{\text{plan}}}$, and is reached by an edge tested at r_{term}. The geodesic distances from S_0 are 51.4, 44.2, 76.2 and 43.6 m, against straight-line distances of 24.0, 41.2, 62.8 and 39.5 m.

5 Shortest Paths

Graph distance (`dijkstra`). The geodesic distance between terminals a and b is the length of the shortest chain of edges between them:

$$\boxed{D_{ab} = \min\Big\{\sum_{l=1}^{L} w_{u_{l-1}u_l} \;:\; u_0 = a,\; u_L = b,\; (u_{l-1}, u_l) \in E\Big\},} \tag{13}$$

with $D_{ab} = \infty$ if no chain exists. The code computes it with Dijkstra's algorithm [2] from each terminal $a = 0, \dots, n$. Starting from $d_a(a) = 0$ and $d_a(v) = \infty$ elsewhere, it repeatedly settles the unsettled node u with the smallest $d_a(u)$ and updates each neighbour v of u:

$$\text{if } d_a(u) + w_{uv} < d_a(v): \qquad d_a(v) \leftarrow d_a(u) + w_{uv}, \qquad \text{pred}_a(v) \leftarrow u. \tag{14}$$

When it stops, every reachable node satisfies

$$d_a(v) = \min_{u:(u,v)\in E} \big(d_a(u) + w_{uv}\big) \quad (v \neq a), \tag{15}$$

and $D_{ab} = d_a(b)$ for the terminals $b = 0, \dots, n$. The code keeps $D = (D_{ab})$, an $n_t \times n_t$ matrix.

The paper's distances. Eq. (9) and Step 1 of Section 0.9 are entries of D:

$$r_i = d_{\text{geo}}(S_0, S_i) = D_{0i}, \qquad D_{\text{geo}} = \big[d_{\text{geo}}(i,j)\big] = \big[D_{ij}\big]_{i,j=1,\dots,n}. \tag{16}$$

In the code these are `geo["D"][0, 1:]` and `geo["D"][1:, 1:]`, with 0-based sensor ids.

Unreachable terminals. If some $D_{ab} = \infty$, `build` stops with the error "some sensors cannot be reached from the base station". `scenario.py` calls it with `require_all=False` instead, to drop candidate sensors that cannot be reached.

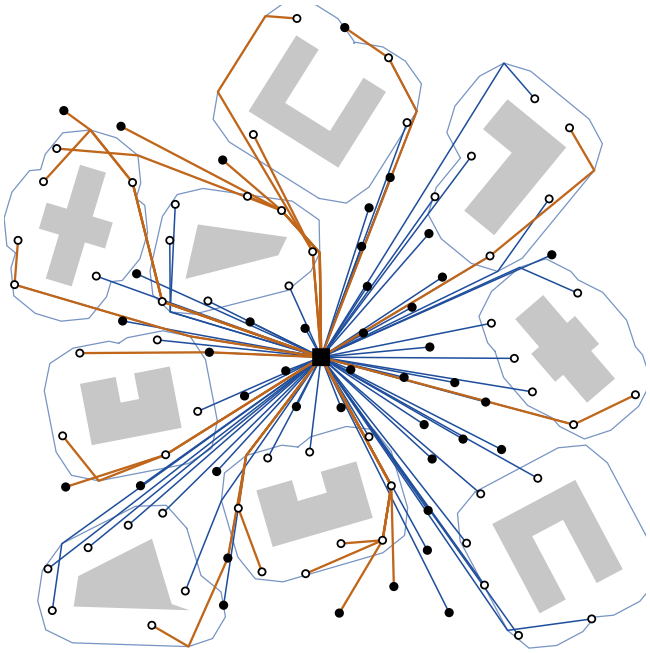


Figure 3: Seed 1: 9 obstacles (grey), their polygons $P_{r_{\text{plan}}}$ (thin blue outline), the base (square) and 98 sensors. The 57 hollow sensors lie inside $P_{r_{\text{plan}}}$. The lines are the 98 paths Γ_{0i} from the base, eq. (17). The 23 orange paths pass through another sensor, where they bend with the margin r_{term}; the blue ones bend only at corners of $P_{r_{\text{plan}}}$. Some paths go round obstacles that reach beyond the 100 m map.

The path (`path`). The shortest path from terminal a to terminal b is read backwards from the predecessors:

$$u_L = b, \qquad u_{l-1} = \text{pred}_a(u_l) \;\text{ until } u_0 = a, \qquad \Gamma_{ab} = \big(v_{u_0}, v_{u_1}, \ldots, v_{u_L}\big). \tag{17}$$

Γ_{ab} is a polyline from S_a to S_b of length D_{ab}, and it is defined only when $D_{ab} < \infty$. `trajectory.py` uses it as the waypoints of a leg.

6 What D Is

A metric. Eq. (13) is a shortest-path distance in an undirected graph with non-negative weights, so

$$D_{aa} = 0, \qquad D_{ab} = D_{ba}, \qquad D_{ac} \le D_{ab} + D_{bc}, \qquad D_{ab} \ge \|S_a - S_b\|, \tag{18}$$

the last because every chain is at least as long as the straight line. On seeds 1–10, D was symmetric and met the triangle inequality to within 6×10^{-14} m, and never fell below the straight-line distance.

Bounds. Among polygonal obstacles, a shortest path between two points is a polyline whose inner vertices are corners of the obstacles [1, 3]. So, if the whole graph were tested at one margin, D would be the true obstacle-avoiding distance at that margin. Here the margin depends on the edge, and D lies between the true distances at the two margins. Write $d(p, q; X)$ for the length of a shortest path from p to q that does not meet the set X. Then

$$\boxed{d\big(S_a, S_b;\; I_{r_{\text{term}}}\big) \;\le\; D_{ab} \;\le\; d\Big(S_a, S_b;\; \text{int}\bigcup_k P_{r_{\text{plan}}}(O_k)\Big),} \tag{19}$$

where the upper bound needs S_a and S_b to lie outside the polygons $P_{r_{\text{plan}}}$. The lower bound holds because every edge is clear at margin r_{term} (an edge clear at r_{plan} is also clear at r_{term}, since $P_{r_{\text{term}}} \subseteq P_{r_{\text{plan}}}$). The upper bound holds because the shortest path that stays out of the polygons $P_{r_{\text{plan}}}$ bends only at their corners, which are nodes, and each of its segments passes the test of eq. (11). On seeds 1–10, both bounds held for every pair, with the true distances computed by single-margin visibility graphs.

Paths through other terminals. A chain in eq. (13) may pass through other terminals, because they are nodes too. Both segments at such a terminal are tested only at r_{term}, so a sensor close to an obstacle can serve as a tighter corner than the corners of $P_{r_{\text{plan}}}$ (Figure 3). On seeds 1–10, 245 of the 1 015 paths from the base pass through another terminal.

What is not guaranteed here. D and Γ_{ab} describe polylines, not the Dubins paths the sinks drive. A clear segment keeps $0.8315\, r_{\text{term}} - \varepsilon \geq 2.08\,\text{m}$ from the obstacles if it touches a terminal, and $0.8315\, r_{\text{plan}} - \varepsilon \geq 5.82\,\text{m}$ if not, but the Dubins path that follows the waypoints can swing wider or cut inside. Eq. (19) of the paper is therefore checked on the final trajectory Γ_k, against the original obstacles that `build` also returns:

$$O = \bigcup_{k=1}^{m} O_k, \qquad \operatorname{dist}\big(\Gamma_k(t), O\big) \geq r_{\text{safe}}. \tag{20}$$

7 Constants

Table 2: Constants in `geodesic.py`, with the defaults of `config.py`.

Constant	Value	Role
$r_{\text{plan}} = r_{\text{safe}} + R$	7 m	inflation for the corner nodes and for lines between two corners
$r_{\text{term}} = r_{\text{safe}} + 0.5$	2.5 m	inflation for lines that start or end at a terminal
q (`quad_segs`)	2	chords per quarter circle, eq. (4)
ε	10^{-6} m	how far a line may enter an inflated polygon, eq. (8)
`require_all`	`True`	stop if a terminal cannot be reached (`scenario.py` passes `False`)

8 Relation to the Paper

The paper states what d_{geo} is (eq. (9): "an obstacle-aware geodesic walking distance in the free space"; Step 1 of Section 0.9: "the shortest obstacle-respecting distance"; and "visibility-based" geometry in the introduction of Section 0.9), but not how it is computed. The code fills the gaps as follows; the README of the implementation records the same choices in its table "Choices where the paper is silent".

Table 3: The paper and `geodesic.py`.

Topic	Paper	`geodesic.py`
Inflation	by r_{safe}, eqs. (4)–(5)	r_{plan} and r_{term} in the graph, eq. (11); r_{safe} on the final path, eq. (20)
Inflated shape	not given	polygon with chords at the corners, eqs. (4)–(7)
How d_{geo} is computed	"visibility-based"	visibility graph and Dijkstra, eqs. (10)–(13)
Sensors close to obstacles	not covered	lines that touch a terminal use r_{term}, eq. (11)
Paths via sensors	not covered	allowed, eq. (13)
Unreachable sensors	not covered	an error, or dropped by `scenario.py`

References

[1] T. Lozano-Pérez and M. A. Wesley, "An algorithm for planning collision-free paths among polyhedral obstacles," Communications of the ACM, vol. 22, no. 10, pp. 560–570, 1979.

[2] E. W. Dijkstra, "A note on two problems in connexion with graphs," Numerische Mathematik, vol. 1, pp. 269–271, 1959.

[3] M. de Berg, O. Cheong, M. van Kreveld and M. Overmars, Computational Geometry: Algorithms and Applications, 3rd ed., Springer, 2008, Chapter 15 (Visibility Graphs).